A Note on "Linear-Communication ACSS with Guaranteed Termination and Lower Amortized Bound"

Xiaoyu Ji
Tsinghua University
jixy23@mails.tsinghua.edu.cn

Junru Li
Tsinghua University
and Shanghai Qi Zhi Institute
jr-li24@mails.tsinghua.edu.cn

Yifan Song
Tsinghua University
and Shanghai Qi Zhi Institute
yfsong@mail.tsinghua.edu.cn

Abstract

In this note, we demonstrate that the construction of asynchronous complete secret sharing (ACSS) proposed in the recent work by Qin et al., accepted at Eurocrypt 2026, is insecure. In particular, we identify several issues affecting the liveness of their protocol and present attacks that compromise its correctness.

1 Introduction

In this note, we point out several security issues in a recent work [QWX26] accepted in Eurocrypt 2026. This note is acknowledged by the authors of [QWX26].

The main contribution of [QWX26] is a new construction for *Asynchronous Complete Secret Sharing* (or ACSS for short) with information-theoretic security in the asynchronous setting with lower communication overhead compared to [JLS24]. Informally, an ACSS protocol allows a dealer to distribute a degree-t Shamir sharing to all parties such that if it succeeds, all honest parties are guaranteed to receive their correct shares.

We note that there are two main challenges in the asynchronous setting:

- First, to achieve liveness, an asynchronous protocol needs to proceed as long as $n - t = 2t + 1$ parties participate. This is because t corrupted parties may never reply.
- Second, there is no restriction on the time order of messages sent by a corrupted party. Even worse, a corrupted party may choose to drop some of its messages.

These two challenges lead to a unique difficulty in the asynchronous setting: In a verification protocol, the random challenge will be generated as long as $2t + 1$ parties are online (to ensure liveness). However, at the moment that a random challenge is generated, there is no guarantee that corrupted parties have sent all previous messages. This is different from the synchronous setting, where in round r, all messages before round r have been sent and delivered. As a result, it opens the door for corrupted parties to generate incorrect messages *after* seeing the random challenge so that the verification still passes. Such an attack would render the verification useless in the asynchronous setting. The security issues in the main construction of [QWX26] are all related to this difficulty, as we elaborate in Section 2.

In addition, we identify additional issues in their other constructions in Section 3.

2 Security Issues in the Main Construction

2.1 A Brief Review of Construction in [QWX26]

At a high level, the construction in [QWX26] follows a similar paradigm to the previous work [JLS24]. Recall that a degree-t Shamir sharing $[x]_t$ is defined by a degree-t polynomial p such that $x = p(0)$ and each party P_i holds a share $p(i)$. Effectively, the goal of an ACSS protocol is to distribute a degree-t polynomial p such that each party P_i obtains $p(i)$.

Following previous works, the idea in [QWX26] is to encode a batch of $t+1$ degree-t polynomials into a random degree-$(t, 2t)$ bivariate polynomial $F(x, y)$ such that $F(x, 0), F(x, -1) \ldots, F(x, -t)$ are those degree-t polynomials to be shared by the dealer. The goal is to let each party P_i learn $F(x, i)$ and $F(i, y)$. In this way, P_i may obtain its shares by evaluating $F(i, 0), F(i, -1), \ldots, F(i, -t)$. In [QWX26], each time the dealer D distributes multiple bivariate polynomials to achieve the claimed amortized communication cost.

Step 1: Commit Shared Bivariate Polynomials. For each $F(x, y)$, D first sends the column polynomial $F(i, y)$ to each party P_i. Then P_i replies to D its signature on $F(i, y)$ via a functionality $\mathcal{F}_{\mathsf{AMC}}$. After receiving signatures from $2t + 1$ parties, D broadcasts the set M consisting of these parties.

Informally, $\mathcal{F}_{\mathsf{AMC}}$ can be viewed as an information-theoretic signature scheme. This functionality is identical to $\mathcal{F}_{\mathsf{APICP}}$ in [JLS24], a generalization of $\mathcal{F}_{\mathsf{AICP}}$ in [PCR09]. $\mathcal{F}_{\mathsf{AMC}}$ allows a signer S to sign a message m to an intermediate I so that later on, I may reveal m to another party R and prove via $\mathcal{F}_{\mathsf{AMC}}$ that this message is indeed from S. In addition, $\mathcal{F}_{\mathsf{AMC}}$ supports revealing a linear combination of signed messages from the same signer S. I.e., with signed messages m_1, m_2 from S, I may reveal a linear combination $c_1 m_1 + c_2 m_2$ to R for publicly agreed coefficients c_1 and c_2.

Notice that the set M contains at least $t + 1$ honest parties, and the column polynomials held by these honest parties uniquely determine $F(x, y)$. Thus, the dealer D effectively commits to the bivariate polynomial $F(x, y)$ to be shared in this step. Later on, when D reveals $F(x, y)$, it needs to also provide valid signatures on $F(i, y)$ for every $P_i \in M$. When multiple bivariate polynomials are committed, since $\mathcal{F}_{\mathsf{AMC}}$ supports revealing a linear combination of signed messages, D can also reveal a linear combination of bivariate polynomials to all parties with publicly agreed coefficients. Using this property, each party P_i, after receiving $F(i, y)$ for each bivariate polynomial $F(x, y)$ from D, may verify the correctness by asking D to reveal a random linear combination of all bivariate polynomials. (To protect the secrecy of $F(x, y)$, this verification requires a random bivariate polynomial as a mask. We omit it for simplicity.)

This step is identical to that in [JLS24] and corresponds to Fig.13 and Fig.14 on Page 25 in [QWX26].

Step 2: Commit Common Points via $\mathcal{F}_{\mathsf{AcceptPoly}}$. Starting from Step 2, the idea in [QWX26] deviates from that in [JLS24]. In this step, the authors make use of a functionality $\mathcal{F}_{\mathsf{AcceptPoly}}$ to let each $P_i \in M$ send the common point $F(i, j)$ to every party P_j such that $F(i, j)$ is consistent with $F(i, y)$ signed by P_i via $\mathcal{F}_{\mathsf{AMC}}$ in Step 1. We note that the description of $\mathcal{F}_{\mathsf{AcceptPoly}}$ in [QWX26] does not correctly capture the security they want to achieve. We will give more discussion about this mismatch in Section 3.1. In the following, we directly plug their realization of $\mathcal{F}_{\mathsf{AcceptPoly}}$ into the main construction.

To achieve this step, each $P_i \in M$ first sends $F(i, j)$ directly to P_j. Then all parties generate a random challenge and ask the dealer to reveal a random linear combination of all bivariate polynomials to P_j. Finally, P_j verifies whether the received $F(i, j)$ matches the bivariate polynomial received from the dealer. One may think that this is a similar verification to that in Step 1, except that only the common point of $F(i, j)$ received from P_i is verified.

This step corresponds to Fig.15 on Page 26 in [QWX26].

Step 3: Reconstructing Row Polynomials. Note that each row of $F(x, y)$ is a degree-t polynomial. After receiving and verifying $F(i, j)$ from $t + 1$ parties in M, each party P_j may locally reconstruct $F(x, j)$. Also note that since there are at least $t + 1$ honest parties in M, each party P_j can receive $F(i, j)$ from at least $t + 1$ parties in M. Thus, every P_j is expected to reconstruct its row polynomial.

This step corresponds to **Reconstructing Row Polynomials** in Fig.16 on Page 27.

Step 4: Reconstructing Column Polynomials. For each P_i, after reconstructing $F(x, i)$, P_i sends $F(j, i)$ to every P_j. Note that P_j may expect at least $2t + 1$ honest parties sending $F(j, i)$ to him, which

is sufficient to reconstruct the degree-$2t$ polynomial $F(j, y)$. However, the issue is that a corrupted P_i may send incorrect points to P_j, leading to incorrect reconstruction of $F(j, y)$.

To solve this, in [QWX26], parties together generate a random challenge and reveal a random linear combination of all shared bivariate polynomials. Then each party P_j verifies $F(j, i)$ from P_i by checking whether it matches the revealed bivariate polynomial. (There are additional steps, including building degree-$(2t, 2t)$ symmetric bivariate polynomials in their construction. However, all these steps are to ensure the correctness of the revealed random linear combination of all shared bivariate polynomials. We omit the details here.)

This step corresponds to **Reconstructing Column Polynomials** in Fig.16 on Page 27.

2.2 Security Issues

Issues in Step 2 and 3. Recall that in Step 2, the authors of [QWX26] wish to let each P_j verify $F(i, j)$ received from P_i. This is done by revealing a random linear combination of committed bivariate polynomials to P_j via $\mathcal{F}_{\mathsf{AMC}}$. However, there are several issues that will lead to failure in their construction:

- First, a corrupted P_i may only send $F(i, j)$ to an honest P_j after the random coefficients are generated. In [QWX26], the random coefficients are generated when $2t + 1$ parties are online, whereas the honest party P_j may not join the protocol yet due to network latency. As a result, parties cannot verify that $F(i, j)$ has been sent to P_j *before* the random coefficients are generated. Then, after learning the random coefficients, the corrupted P_i may send incorrect $F(i, j)$, but the linear combination matches the revealed bivariate polynomial.

- Second, a corrupted dealer D may never send the random linear combination of all bivariate polynomials to P_j when $\mathcal{F}_{\mathsf{AMC}}$ is invoked, causing P_j to wait forever. We note that this issue can be easily addressed by asking D to broadcast this bivariate polynomial to all parties instead.

One may attempt to address the first issue by generating the random coefficients only after P_j receives $F(i, j)$ from P_i, say P_j generates the random coefficients himself and broadcasts them after $F(i, j)$ is received. While this avoids the aforementioned attack, it leads to a termination issue even when the dealer is honest: The revelation of the random linear combination of all bivariate polynomials is done by $\mathcal{F}_{\mathsf{AMC}}$, which requires D to be online. However, in the asynchronous setting, it is possible that $2t + 1$ parties, including the dealer, have terminated the execution while P_j joins the execution only afterwards. Then, P_j may never receive any response from D if the challenges are generated after P_j is online, leading to waiting forever.

Due to the above issues, there is no guarantee that every honest party P_j can reconstruct its row polynomial $F(x, j)$ anymore.

Issue in Step 4. We note that even omitting the above issues and assuming that parties can receive their correct row polynomials, Step 4 in [QWX26] does not ensure reconstruction of each party's column polynomial either.

Recall that in Step 4, a party P_j needs to verify $F(j, i)$ received from P_i and only uses correct points to reconstruct $F(j, y)$. However, the verification is done by letting all parties reconstruct a random linear combination of all shared bivariate polynomials. Then the same problem occurs: a corrupted P_i may only send $F(j, i)$ to an honest P_j after the random coefficients are generated. We give a concrete attack below that causes honest parties to terminate with incorrect degree-t Shamir shares.

Suppose the first t parties are corrupted. Let $A := \{P_{t+2}, \ldots, P_{2t+1}\}$ and $B := \{P_{2t+2}, \ldots, P_{3t+1}\}$. Suppose the dealer $D \notin B$ wishes to share $F_1(x, y)$ and $F_2(x, y)$. The adversary honestly follows the protocol but blocks the messages sent to or sent by parties in B until honest parties in $A \cup \{P_{t+1}\}$ terminate. Effectively, honest parties in B only join the execution after honest parties in $A \cup \{P_{t+1}\}$ terminate. Following the construction in [QWX26], the random challenge d in Step 3.(4) has been generated, and all parties have broadcast $F(x, y) := d \cdot F_1(x, y) + d^2 \cdot F_2(x, y)$ (here we omit the random mask for simplicity). Then the adversary does the following:

1. Let $\Delta(x, y)$ be an arbitrary non-zero degree-$(t, 2t)$ bivariate polynomial such that the row polynomial of every party in $B \cup \{P_{t+1}\}$ is all-0, and the column polynomial of P_{t+1} is all-0. Such a bivariate polynomial can be found by setting $\Delta(t+1, y) \equiv 0$ and setting $\Delta(i, y)$ to be an arbitrary non-zero degree-$2t$ polynomial for all $i \in [2t+2, 3t+1]$ such that $\Delta(i, j) = 0$ for all $j = t+1$ and $j \in [2t+2, 3t+1]$. Then $\Delta(x, y)$ interpolated from these $t+1$ column polynomials satisfies the requirement.

   Let $F_1'(x, y) := F_1(x, y) + d \cdot \Delta(x, y)$ and $F_2'(x, y) := F_2(x, y) - \Delta(x, y)$. Then

   $$F(x, y) = d \cdot F_1(x, y) + d^2 \cdot F_2(x, y) = d \cdot F_1'(x, y) + d^2 \cdot F_2'(x, y).$$

   In addition, P_{t+1}'s column and row polynomials are identical in $(F_1(x, y), F_2(x, y))$ and $(F_1'(x, y), F_2'(x, y))$. For each party in B, its row polynomials are identical in $(F_1(x, y), F_2(x, y))$ and $(F_1'(x, y), F_2'(x, y))$ as well.

2. When $d \neq 0$, which happens with overwhelming probability, each corrupted party P_i sends $F_1'(j, i)$ and $F_2'(j, i)$ to every honest party $P_j \in B$. Then the adversary continues to block the messages sent from parties in A to parties in B, but delivers the rest of the messages until honest parties in B terminate. Effectively honest parties in B will never receive any message from honest parties in A before termination.

Note that honest parties in B join the computation only after honest parties in $A \cup \{P_{t+1}\}$ terminate. In the above attack, all honest parties will receive their correct row polynomials. In Step 4, however, honest parties in B only receive points from parties outside of A. While they will also receive the broadcast bivariate polynomial $F(x, y)$, by the choice of $F_1'(x, y), F_2'(x, y)$, parties in B will accept these points and reconstruct their column polynomials. As a result, each party $P_j \in B$ would take $(F_1'(j, y), F_2'(j, y)) \neq (F_1(j, y), F_2(j, y))$ as output, and their shares are not consistent with those held by parties in A.

Remark 1 *One may attempt to address this issue by generating the random coefficients after P_j receives $F_1(j, i), F_2(j, i)$ from P_i. Again, this leads to the termination issue that $2t+1$ parties may have terminated the protocol and never respond, leading to waiting forever for parties in B.*

3 Security Issues in Other Sub-protocols

3.1 Mismatch Between $\mathcal{F}_{\mathsf{AcceptPoly}}$ and Its Realization

In [QWX26], $\mathcal{F}_{\mathsf{AcceptPoly}}$ takes as input m vectors $\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m$, each of size n, from a sender S. Then for an acceptor P_i, $\mathcal{F}_{\mathsf{AcceptPoly}}$ delivers $s_{1,i}, \ldots, s_{m,i}$ to P_i. (In their construction, $\mathcal{F}_{\mathsf{AcceptPoly}}$ actually sends a polynomial whose coefficients are $s_{1,i}, \ldots, s_{m,i}$. This is equivalent to sending those values directly.) However, when S is corrupted, $\mathcal{F}_{\mathsf{AcceptPoly}}$ allows S to not send any message to P_i or change the messages arbitrarily. We note that such a functionality can be easily realized by just asking S to send $s_{1,i}, \ldots, s_{m,i}$ to P_i.

After discussions with the authors and checking their realization of $\mathcal{F}_{\mathsf{AcceptPoly}}$, what they meant to realize is to ensure the following:

- In their main construction of ACSS, S will receive its column polynomials from D and reply its signature via $\mathcal{F}_{\mathsf{AMC}}$. The messages $\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m$ in $\mathcal{F}_{\mathsf{AcceptPoly}}$ correspond to column polynomials of S from m bivariate polynomials.
- They want to ensure that $s_{1,i}, \ldots, s_{m,i}$ sent to P_i are consistent with $\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m$ signed by S via $\mathcal{F}_{\mathsf{AMC}}$.

As evidence, their realization of $\mathcal{F}_{\mathsf{AcceptPoly}}$ in Fig. 11 on Page 23 incurs invocations of $\mathcal{F}_{\mathsf{AMC}}$, which are not initialized and unspecified parameters $\boldsymbol{c}$. These are only meaningful when plugging their realization of $\mathcal{F}_{\mathsf{AcceptPoly}}$ into the main construction.

3.2 Security Issues in Realization of $\mathcal{F}_{\mathsf{AMC}}$

Recall that $\mathcal{F}_{\mathsf{AMC}}$ is identical to $\mathcal{F}_{\mathsf{APICP}}$ in [JLS24]. The authors in [QWX26] propose a more efficient construction compared to the results in [JLS24], but there exist several security issues.

Review of $\mathcal{F}_{\mathsf{AMC}}$. The functionality $\mathcal{F}_{\mathsf{AMC}}$ can be viewed as a signature scheme involving three parties: a signer Sig, an intermediary I, and a receiver R. The protocol consists of two phases: the **Init** phase and the **Rev** phase. In the **Init** phase, the signer Sig signs a message and sends it to I. Upon completing the **Init** phase, I must be able to forward the message received from Sig to the receiver R, and prove that the message indeed originates from Sig, during the **Rev** phase. Importantly, in the **Rev** phase, I cannot rely on the signer Sig being online to assist with this proof.

Concretely speaking, the AMC protocol should require the following:

- If both the signer Sig and the intermediary I are honest, then we guarantee the termination of the **Init** phase. During the **Rev** phase, an honest R is guaranteed to accept the message received from I.
- If the signer Sig is honest and signs a message m for I, a corrupted intermediary I *cannot* cause an honest R to accept any message $m' \neq m$.
- If the intermediary I is honest and terminates the **Init** phase with message m, then it must guarantee that an honest R will always accept the message m, even if the signer Sig is corrupted.
- If both the signer Sig and the intermediary I are corrupted, no security guarantee is required.

In addition to the above requirements, $\mathcal{F}_{\mathsf{AMC}}$ also allows the intermediary I to reveal linear combinations of the signer Sig's messages, and such revelations can be performed multiple times.

Review of the Construction in [QWX26]. We use $\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m$ to denote the signer Sig's input, where m is the batch size and each $\boldsymbol{s}_\ell$ is of size n field elements for all $\ell \in [m]$. In the **Init** phase, the goal is to let Sig send these $\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m$ to I, and allow I to reveal a linear combination of these vectors to R multiple times in the **Rev** phase, where the receiver R can be different in different revelation procedures.

At a high-level, the authors in [QWX26] aim to generate a MAC in the form of $(\boldsymbol{\alpha}, \tau_\ell)$ such that $\tau_\ell = \langle \boldsymbol{\alpha}, \boldsymbol{s}_\ell \rangle$ for each vector $\boldsymbol{s}_\ell, \ell \in [m]$ in the **Init** phase. Note that $\boldsymbol{\alpha} \in \mathbb{F}^n$ is reused for all $\ell \in [m]$. To realize this step, they first let the signer Sig randomly sample $\boldsymbol{\alpha}$ and compute $\tau_\ell = \langle \boldsymbol{\alpha}, \boldsymbol{s}_\ell \rangle$. Then Sig will distribute the degree-t sharings $\{[\tau_\ell]_t\}_{\ell \in [m]}$ and $[\boldsymbol{\alpha}]_t$ to all parties. Intuitively, if Sig is honest, then these MACs authenticate Sig's messages $\{\boldsymbol{s}_\ell\}_{\ell \in [m]}$. In the **Rev** phase, to verify the correctness of I's messages $\boldsymbol{s}' = \sum_{\ell \in [m]} c_\ell \cdot \boldsymbol{s}'_\ell$, where $\{c_\ell\}_{\ell \in [m]}$ are publicly agreed coefficients, parties can reconstruct the MAC $(\boldsymbol{\alpha}, \tau := \sum_{\ell \in [m]} c_\ell \cdot \tau_\ell)$ to the receiver R, and R checks whether $\tau = \langle \boldsymbol{\alpha}, \boldsymbol{s}' \rangle$. Since the MACs remain unknown to I when R is honest, a corrupted I cannot let R accept $\boldsymbol{s}' \neq \sum_{\ell \in [m]} c_\ell \cdot \boldsymbol{s}_\ell$ with an overwhelming probability if the field size is large enough.

One thing to note is that the above solution does not support multiple revelations: If R is corrupted, then it can learn $\boldsymbol{\alpha}$ during the **Rev** phase. For the next time of **Rev** phase for an honest R, the corrupted I can first find a non-zero $\boldsymbol{e}$ such that $\langle \boldsymbol{\alpha}, \boldsymbol{e} \rangle = 0$ and then find $\{\boldsymbol{e}_\ell\}_{\ell \in [m]}$ such that $\boldsymbol{e} = \sum_{\ell \in [m]} c_\ell \cdot \boldsymbol{e}_\ell$. Later, it can let an honest party R accept $\boldsymbol{s}' = \sum_{\ell \in [m]} c_\ell \cdot (\boldsymbol{s}_\ell + \boldsymbol{e}_\ell) \neq \sum_{\ell \in [m]} c_\ell \cdot \boldsymbol{s}_\ell$ since the following relation holds.

$$\langle \boldsymbol{\alpha}, \boldsymbol{s}' \rangle = \langle \boldsymbol{\alpha}, \sum_{\ell \in [m]} c_\ell \cdot \boldsymbol{s}_\ell \rangle + \langle \boldsymbol{\alpha}, \sum_{\ell \in [m]} c_\ell \cdot \boldsymbol{e}_\ell \rangle = \tau + \langle \boldsymbol{\alpha}, \boldsymbol{e} \rangle = \tau.$$

To support multiple revelations for different R, the authors in [QWX26] use the trick of the Beaver triple technique. Concretely, for each revelation, they let Sig distribute a triple in the form of $([\boldsymbol{a}]_t, [\boldsymbol{b}]_t, [c]_t)$ such that $c = \langle \boldsymbol{a}, \boldsymbol{b} \rangle$. Then, during the **Rev** phase, instead of directly revealing the MAC $(\boldsymbol{\alpha}, \tau)$ to R, parties do the following steps.

1. I distributes the sharings $[\boldsymbol{s}]_t$ to all parties.

2. All parties compute their shares of $[\Delta\boldsymbol{\alpha}]_t = [\boldsymbol{\alpha}]_t - [\boldsymbol{a}]_t$, $[\Delta\boldsymbol{s}]_t = [\boldsymbol{s}]_t - [\boldsymbol{b}]_t$, then they reconstruct the secrets $\Delta\boldsymbol{\alpha}, \Delta\boldsymbol{s}$, and forward them to R.

3. All parties compute their shares of $[v']_t = [c]_t + \langle \Delta\boldsymbol{\alpha}, [\boldsymbol{b}]_t \rangle + \langle \Delta\boldsymbol{s}, [\boldsymbol{a}]_t \rangle, [\tau]_t = \sum_{\ell \in [m]} c_\ell \cdot [\tau_\ell]_t$, and reconstruct the secrets v', τ to R.

4. Upon receiving the same $\Delta\boldsymbol{\alpha}, \Delta\boldsymbol{s}$ from $2t+1$ parties and reconstructing the secrets v', τ, R verifies whether $\tau = v' + \langle \Delta\boldsymbol{\alpha}, \Delta\boldsymbol{s} \rangle$.

Remark 2 *We note that there are additional steps (from Step 5 to 11) in their construction of the* ***Init*** *phase (Fig. 6) that appear to be useless: These steps do not involve any verification of the MACs (generated in Step 2 and 3) and triples (generated in Step 4) that are used in the* ***Rev*** *phase, and all new variables introduced in these steps are never used in the* ***Rev*** *Phase. We ignore these steps in the following security analysis.*

Security Issues. The first issue is that the reconstruction of $[\Delta\boldsymbol{\alpha}]_t, [\Delta\boldsymbol{s}]_t, [v']_t, [\tau]_t$ may never terminate when Sig is corrupted. This is because in the **Init** phase, Sig directly distributes degree-t Shamir sharings to all parties. However, these are not ACSS and it does not guarantee that honest parties eventually receive correct shares. As a result, the online error correction ($\mathcal{F}_{\mathsf{PrivRec}}$ in [QWX26]) would not work. Then when Sig is corrupted but I is honest, even if the **Init** phase terminates, the **Rev** phase may never terminate.

The second issue is that if Sig is corrupted, then if all parties terminate the **Init** phase, an honest I cannot guarantee that an honest R can accept its message. This is because a corrupted D can distribute incorrect MACs $(\tau'_\ell, \boldsymbol{\alpha})$ such that $\tau'_\ell \neq \langle \boldsymbol{\alpha}, \boldsymbol{s}_\ell \rangle$ for some $\ell \in [m]$ and incorrect triples $([\boldsymbol{a}']_t, [\boldsymbol{b}']_t, [c']_t)$ such that $c' \neq \langle \boldsymbol{a}', \boldsymbol{b}' \rangle$. In their construction of the **Init** phase, there is no verification that prevents a corrupted D from launching these attacks. With incorrect verification messages, there is no hope of letting an honest R succeed in the verification.

3.3 Other Issues

Issue in Π_{PrivRec}. The construction of the AMC protocol in [QWX26] relies on the functionality $\mathcal{F}_{\mathsf{PrivRec}}$ for the online error correction (OEC) process. In an OEC process, a receiver reconstructs the secret of a complete secret sharing $[x]_t$ (where every P_i holds $p(i)$ for a degree-t polynomial p with $p(0) = x$) by letting all the parties send their shares, where the corrupted parties' shares may be dropped or sent incorrectly. The idea of OEC is that the receiver can always expect to receive $2t+1$ correct shares on the sharing such that the honest parties' shares among them determine the secret. Thus, after receiving $2t+1+r$ shares, the receiver can check whether fewer than r errors exist among them using Reed–Solomon decoding (e.g., via Berlekamp–Welch [WB86]). If not, the receiver knows that some honest party's share does not arrive, so it can wait for more shares.

Although the OEC process is standard, the provided protocol Π_{PrivRec} (in Section D.1 of [QWX26]) that realizes $\mathcal{F}_{\mathsf{PrivRec}}$ is problematic. The idea of [QWX26] of detecting errors after receiving $2t+1+r$ shares is that:

1. The receiver randomly chooses two different sets $\mathcal{W}_1, \mathcal{W}_2$ of $2t+1$ parties and reconstructs two degree-t polynomials $\hat{p}(\cdot), \tilde{p}(\cdot)$ via Lagrange interpolation.

2. If $\hat{p}(\cdot) = \tilde{p}(\cdot)$, the receiver reconstructs $\hat{p}(0)$ as the secret. Otherwise, the receiver waits for more shares to arrive.

Consider the case that $P_1, \ldots, P_{2t+1}$ are honest and the other parties are corrupted, and they hold the complete secret sharing $[0]_t = (0, 0, \ldots, 0)$ that needs to be reconstructed to an honest receiver. When $2t+2$ shares arrive, where $2t+1$ of them are sent by the honest parties, and the other share is an incorrect share 1 sent by P_{2t+2}. The receiver, with a noticeable probability, may select $\mathcal{W}_1 = \{1, \ldots, 2t, 2t+2\}$ and $\mathcal{W}_2 = \{2, 3, \ldots, 2t+2\}$. In this case, we have $\hat{p}(x) = \prod_{k=1}^{2t} \frac{x-\alpha_k}{\alpha_{2t+2}-\alpha_k}$ and $\tilde{p}(x) = \prod_{k=2}^{2t+1} \frac{x-\alpha_k}{\alpha_{2t+2}-\alpha_k}$ which are

different. Therefore, the receiver will wait until more shares arrive following the protocol. However, still in this case, the parties $P_{2t+3}, P_{2t+4}, \ldots, P_{3t+1}$ are all corrupted, and they may never send their shares, so the reconstruction process will never terminate.

Issue in the Communication Cost of Random Coins. In [QWX26], the authors use the lemma in [JLS24] that the amortized cost of generating each random public coin is $O(n^3\kappa)$ to analyze their total communication cost. However, the authors do not consider the overhead of generating these coins, which may exceed the claimed overhead $O(n^2\kappa^2 + n^5\kappa)$ of the ACSS protocol in [QWX26].

References

[JLS24] Xiaoyu Ji, Junru Li, and Yifan Song. Linear-communication asynchronous complete secret sharing with optimal resilience. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VIII*, volume 14927 of *LNCS*, pages 418–453. Springer, Cham, August 2024.

[PCR09] Arpita Patra, Ashish Choudhary, and C Pandu Rangan. Efficient statistical asynchronous verifiable secret sharing with optimal resilience. In *International Conference on Information Theoretic Security*, pages 74–92. Springer, 2009.

[QWX26] Chengyi Qin, Mingqiang Wang, and Haiyang Xue. Linear-communication ACSS with guaranteed termination and lower amortized bound. Cryptology ePrint Archive, Paper 2026/304, 2026.

[WB86] Lloyd R Welch and Elwyn R Berlekamp. Error correction for algebraic block codes, December 30 1986. US Patent 4,633,470.